

Week 3 Monday Lab Solution:

Q1. Source – Where pipeline finds data

Ingestion – Extract data from source to workflow

Raw - Stores data as raw data

Staging – Apply cleaning

Transformation – Transform data in the required form according to business purpose etc

Analytics – Analyze data

Q2.

Area

ETL

ELT

When transformation happens

After ingestion of data

After loading data

Where transformation usually runs

After extracting in a script

Where database is

Suitable example

Python code to from a .csv file

A SQL code to transform it

Q3. Given Flow

Extract JSON files

↓

Clean and standardise with Python

↓

Load cleaned rows into the warehouse

Ans:

--- Original flow is ETL

----Extract JSON files

↓

Load data into the warehouse

↓

Clean and standardise with SQL

----- Transformation will happen in the warehouse

Q4

--ETL

--Before loading data can be cleaned for inconsistent date formats and invalid price

--Dates and price can be standardized also

-- Any transformation can be processed externally by outsourcing

Q5- What is the main purpose of a data pipeline?

B- Move and process data through defined stages

Q6 – Which statement best describes a raw layer?

B - It stores data close to its original form

Q7 – Which sequence represents ETL?

A - Extract → Transform → Load

Q8 – Where does transformation usually happen in ELT?

B - After loading into the target platform

Q9 - Which scenario most strongly supports ELT?

A - A modern warehouse can store raw data and run large SQL transformations

Q10:

Use case

Choice

Daily sales report

Batch, once in a day it can report batch wise

Payment-fraud alert

Streaming – It has to be done instantly

Hourly stock snapshot

Batch, it can runs hourly

Immediate delivery notification

Streaming- it has to be active real time

Q11. Simulate Batch and Streaming Events

```
# Q11 . Write one Python script that:  
  
# Creates five order events.  
#Prints all five together as a batch.  
#Prints the same five one at a time with a short delay.  
#Labels both sections clearly.
```

[1] ✓ 0.0s

```
import pandas as pd  
import numpy as np  
import time  
from datetime import datetime
```

[7] ✓ 0.0s

```
# Create Five Order events  
orders = [  
    {"order_id": 1, "amount_euro": 24.99},  
    {"order_id": 2, "amount_euro": 50.50},  
    {"order_id": 3, "amount_euro": 10.75},  
    {"order_id": 4, "amount_euro": 120.00},  
    {"order_id": 5, "amount_euro": 200.20},  
]
```

[8] ✓ 0.0s

```
#Prints all five together as a batch.  
print("BATCH")  
for order in orders:  
    print(order)  
  
#Prints the same five one at a time with a short delay.  
  
print("\nSTREAMING SIMULATION")  
for order in orders:  
    print(datetime.now().isoformat(timespec="seconds"), order)  
    time.sleep(1)
```

[9] ✓ 5.0s

```
... BATCH  
{'order_id': 1, 'amount_euro': 24.99}  
{'order_id': 2, 'amount_euro': 50.5}  
{'order_id': 3, 'amount_euro': 10.75}  
{'order_id': 4, 'amount_euro': 120.0}  
{'order_id': 5, 'amount_euro': 200.2}  
  
STREAMING SIMULATION  
2026-07-20T15:18:51 {'order_id': 1, 'amount_euro': 24.99}  
2026-07-20T15:18:52 {'order_id': 2, 'amount_euro': 50.5}  
2026-07-20T15:18:53 {'order_id': 3, 'amount_euro': 10.75}  
2026-07-20T15:18:54 {'order_id': 4, 'amount_euro': 120.0}  
2026-07-20T15:18:55 {'order_id': 5, 'amount_euro': 200.2}
```

Is to analyze

Q12: Ingest a Customer File into Raw Storage

customer_id,name,email,city

1, Amelia Clarke ,AMELIA@EXAMPLE.COM,Manchester

2,Oliver Bennett,oliver@example.com,Bristol

```

# Q12: Create data/source/customers.csv with the supplied data.

import os
import shutil
import csv

# Create folders
os.makedirs("data/source", exist_ok=True)
os.makedirs("data/raw", exist_ok=True)

# Create source file with supplied data
data = """customer_id,name,email,city
1, Amelia Clarke ,AMELIA@EXAMPLE.COM,Manchester
2,Oliver Bennett,oliver@example.com,Bristol
"""

with open("data/source/customers.csv", "w", newline="") as f:
    f.write(data)

# Copy unchanged to data/raw
shutil.copyfile("data/source/customers.csv", "data/raw/customers.csv")

# Count copied data rows (excluding header)
with open("data/raw/customers.csv", newline="") as f:
    reader = csv.reader(f)
    row_count = sum(1 for _ in reader) - 1

print(f"Number of copied data rows: {row_count}")

```

✓ 0.0s

Python

Number of copied data rows: 2

Q13: Clean Raw Customers into Staging

```
# Q13: Clean Raw Customers into Staging

# Create staging folder
os.makedirs("data/staging", exist_ok=True)

cleaned_rows = []

with open("data/raw/customers.csv", newline="") as f:
    reader = csv.DictReader(f)
    fieldnames = reader.fieldnames
    for row in reader:
        row["name"] = row["name"].strip()
        row["email"] = row["email"].strip().lower()
        cleaned_rows.append(row)

with open("data/staging/customers.csv", "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=fieldnames)
    writer.writeheader()
    writer.writerows(cleaned_rows)

# Print cleaned rows
for row in cleaned_rows:
    print(row)
```

[15] ✓ 0.0s Python

```
... {'customer_id': '1', 'name': 'Amelia Clarke', 'email': 'amelia@example.com', 'city': 'Manchester'}
    {'customer_id': '2', 'name': 'Oliver Bennett', 'email': 'oliver@example.com', 'city': 'Bristol'}
```

Q14. Which processing style best fits a report produced once every night?

A- Batch

Q15- Which processing style best fits an immediate fraud alert?

B- Streaming

Q16- What does an ingestion job normally do?

A. Moves data into a pipeline

Q17- Which task belongs most naturally in staging?

A. Trim names and validate required fields

Q18- Why keep raw and staging data separate?

A. To preserve the original input while creating a cleaned version

Q19:

--- ingest_customers

↓

clean_customers

↓

build_customer_summary

↓

publish_report

---- If clean_customer fails then the next step build_customer_summary will not be performed.

---- Because each step depends on the previous one.

Q20 :

```
#Q. 20 : Stop a Pipeline after Failure

#Create a Python runner with these functions:
#ingest() returns success.
#clean() raises FileNotFoundError when data/raw/customers.csv is missing.
#publish() runs only when earlier functions succeed.
#The runner prints the failed task name and stops.

from pathlib import Path

def ingest():
    print("Ingestion has done successfully")

def clean():
    raw_file = Path("data/raw/customers.csv")
    if not raw_file.exists():
        raise FileNotFoundError(raw_file)
    print("cleaned")

def publish():
    print("publish:success")

def run_pipeline():
    tasks = [
        ("ingest", ingest),
        ("clean", clean),
        ("publish", publish),
    ]

    for task_name, task_func in tasks:
        try:
            task_func()
        except Exception as e:
            print(f"Task failed: {task_name} ({e})")
            print("Pipeline stopped.")
            return # stop the pipeline here

    print("Pipeline completed successfully.")

if __name__ == "__main__":
    run_pipeline()
```

[20] ✓ 0.0s

```
... Ingestion has done successfully
    cleaned
    publish:success
    Pipeline completed successfully.
```

Q21:

```
from pathlib import Path
import hashlib
import shutil

source = Path("data/source/customers.csv")
target = Path("data/raw/customers.csv")
target.parent.mkdir(parents=True, exist_ok=True)

def file_hash(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as file:
        for chunk in iter(lambda: file.read(8192), b""):
            digest.update(chunk)
    return digest.hexdigest()

if target.exists() and file_hash(source) == file_hash(target):
    print("Ingestion skipped: raw file is already current")
else:
    shutil.copyfile(source, target)
    print("Ingestion complete: raw file copied")
```

[21] ✓ 0.0s

```
... Ingestion skipped: raw file is already current
```